

Security Assessment Report

Reporting period: **2026-08** · Generated: 2026-08-23T02:22:39.993Z

Executive Summary

This report summarizes the periodic security assessment of the crew scheduling system web application and infrastructure (SIT / UAT). 43 findings were identified across nmap, nuclei, pentagi, zap scanning engines.

Severity	Count
critical	0
high	4
medium	14
low	18
info	7

Severity breakdown: 4 high, 14 medium, 18 low, 7 info

Open findings: 31 · **Critical/High:** 4 · **OWASP 2021 categories touched:** A03 · **PCI DSS requirements touched:** 6.5.7

Scope & Methodology

Targets (allowlist):

- https://crew-f8-usva-sit.roiscloud.com/live/
- https://crew-f8-usva-sit.roiscloud.com/pbs/
- https://crew-f8-usva-sit.roiscloud.com/altair/
- https://crew-f8-usva-sit.roiscloud.com/report/
- https://crew-f8-usva-sit.roiscloud.com/rule/
- https://crew-f8-usva-sit.roiscloud.com/engine/
- https://crew-f8-usva-uat.roiscloud.com/live/
- https://crew-f8-usva-uat.roiscloud.com/pbs/
- https://crew-f8-usva-uat.roiscloud.com/altair/
- https://crew-f8-usva-uat.roiscloud.com/report/
- https://crew-f8-usva-uat.roiscloud.com/rule/
- https://crew-f8-usva-uat.roiscloud.com/engine/

Engines: nmap, nuclei, pentagi, zap · **Frequency:** weekly scan, monthly report · **Environment:** SIT and UAT only (production is excluded).

Coverage scope. Web application layer — all allowlisted routes under the SIT/UAT domains, assessed against the OWASP Top 10 (2021), CWE/SANS Top 25, and PCI DSS 6.5.x. Network & infrastructure layer — perimeter service discovery over common web and management ports (22, 80, 443, 8443). AI-assisted deep testing of high-value endpoints. Production, non-allowlisted hosts and ports, and third-party infrastructure are explicitly out of scope.

Assessment Framework & Methodology

This assessment is executed with an industry-standard, OWASP-aligned toolchain. Every engine is open-source and widely adopted across enterprise security teams and public bug-bounty programs; together they cover the majority of commonly exploited vulnerability classes across both the application and the network infrastructure layers. Each engine is described below.

OWASP ZAP — Dynamic Web Application Security Testing (DAST)

Industry Standing. OWASP flagship open-source dynamic application security testing (DAST) tool, maintained under the Linux Foundation Software Security Project. It is the de-facto standard for free web-application scanning, widely deployed in enterprise security teams, CI/CD pipelines and bug-bounty programs.

Detection Coverage. Operates as an intercepting proxy with full visibility into client-server traffic. The passive scanner analyses every request and response without sending additional traffic, flagging missing security headers (X-Frame-Options, CSP), weak cookie flags (Secure, HttpOnly), information disclosure and TLS misconfiguration. The active scanner fires real attack payloads across policy families - injection, client-side scripting (XSS), server security, and information disclosure - to confirm exploitable flaws.

Method. The spider and AJAX spider (a headless-browser crawler) build the application map, including JavaScript-driven single-page routes that never appear in raw HTML; passive rules run continuously on proxied traffic; the active scanner mutates requests against allowlisted routes. Driven through the Automation Framework (versionable YAML plans) via `zap.sh -cmd -autorun`; findings are normalized into the unified finding model.

Nuclei — Template-Driven Vulnerability Scanning

Industry Standing. ProjectDiscovery open-source, Go-based, template-driven vulnerability scanner, adopted by Fortune 500 enterprises, government agencies, bug-bounty platforms and DevSecOps teams. Security checks are shared as small YAML templates rather than compiled plugins, so coverage is community-vetted and continuously updated.

Detection Coverage. A community-curated template library of 10,000+ checks spanning known CVEs and exposures, OWASP Top 10 web flaws, misconfigurations, exposed services and cloud infrastructure issues. Templates are tagged and filterable by severity, technology and CWE, and include CISA KEV-tagged templates for actively exploited vulnerabilities.

Method. Each YAML template encodes a security test as metadata plus protocol-specific requests (HTTP, DNS, TCP, SSL) with word/regex matchers and dynamic extractors. Rather than relying on version banners, templates probe the real interface an attacker would hit and confirm a finding with reproducible response evidence, materially reducing false positives.

Nmap — Network Discovery & Security Auditing

Industry Standing. The de-facto standard for network discovery and security auditing since 1997, used by system administrators, penetration testers and compliance auditors worldwide.

Detection Coverage. Host discovery, TCP/UDP port scanning, service and version detection (identifying the exact software and version so outdated or misconfigured components can be flagged), operating-system fingerprinting, and scriptable checks through the Nmap Scripting Engine (NSE), including CVE and vulnerability-category scripts.

Method. Port scanning and service/version detection run over the allowlisted perimeter ports (22, 80, 443, 8443); NSE default and vulnerability scripts add service enumeration and known-vulnerability checks. Findings are normalized into the unified finding model alongside the web engines.

PentAGI — AI-Assisted Deep Testing

Industry Standing. Open-source, fully autonomous AI-agent penetration-testing platform (vxcontrol). A multi-agent system in which specialist agents - orchestrator, pentester, coder, researcher and adviser - collaborate on complex multi-step security testing tasks, with support for multiple LLM providers.

Detection Coverage. AI-driven deep-dive testing of high-value endpoints and business flows that automated scanners cover poorly: multi-stage attack simulation, exploit-chain development, reconnaissance and CVE research, and business-logic abuse checks.

Method. Agents plan and execute multi-step attack sequences inside a sandboxed environment, with tool access permission-wrapped per agent role. A human-in-the-loop flow reviews the final vulnerability register before results are normalized into the unified finding model and AI-triaged to filter false positives.

Detection & verification workflow. Weekly automated scans run against the SIT/UAT allowlist (production is excluded). Results from all engines are normalized into a single finding model, mapped to OWASP 2021 and PCI DSS, and AI-triaged to filter false positives before confirmed findings are tracked as issues. Each fix is recorded with evidence — commit reference, code snippet, and screenshots — and re-verified by the next scan cycle: findings that no longer reproduce are marked verified, while regressions are automatically reopened for follow-up.

Assessment Coverage by Scenario

The sections below map the penetration-testing scenarios covered by this assessment to what is checked in each scenario and which part of the toolchain covers it. Scenarios that require a dedicated engagement — such as mobile or database assessments, social engineering, and physical security — are outside this recurring automated assessment and are not listed.

Web Application Vulnerabilities

What Is Checked. All allowlisted routes and API endpoints are assessed against the OWASP Top 10 (2021):

- Broken access control and insecure direct object references
- Cryptographic failures
- Injection - SQL injection and NoSQL, OS command, LDAP, template injection and cross-site scripting (XSS)
- Insecure design
- Security misconfiguration (including XXE)
- Vulnerable and outdated components
- Identification and authentication failures
- Software and data integrity failures (unsafe deserialization)
- Security logging and monitoring failures
- Server-side request forgery

Framework Coverage. Automated. OWASP ZAP runs passive and active scanning over allowlisted routes; Nuclei matches CVE, misconfiguration and exposure templates; PentAGI performs targeted deep tests of high-value endpoints. This is the core strength of the weekly scan.

Network & System Host Vulnerabilities

What Is Checked. Perimeter services and host operating systems are checked for:

- Weak or default credentials on SSH, Telnet, FTP, terminal and IIS services
- Overflow and privilege-escalation flaws in operating systems and application services
- Configuration defects - IIS permission misconfiguration, SNMP read/write exposure, TFTP configuration extraction and insecure encryption
- Exposed management or debug services
- TLS and cipher misconfiguration

Framework Coverage. Partially automated. Nmap performs port, service, version and OS discovery and fingerprinting over the allowlisted perimeter (22, 80, 443, 8443), with NSE script checks for known weaknesses. Credential and privilege-escalation testing is not automated and is handled by manual review or an on-demand engagement.

Business Logic & Access Control Vulnerabilities

What Is Checked. Application business flows and authorization boundaries are checked for:

- Malicious registration, account-takeover risk, payment and transaction security, flash-sale or ranking abuse, and spam
- Verification of authentication and session-management modules
- Horizontal and vertical privilege escalation
- Insecure direct object references (IDOR) and broken object-level authorization
- Cross-site request forgery (CSRF)
- File and database interface controls to prevent unauthorized modification of data or access to sensitive files

Framework Coverage. Partially automated. PentAGI runs AI-assisted deep tests of high-value endpoints and business flows; automated scanners have limited logic coverage, so business-logic verification is supplemented by manual review.

System Architecture & Deep-Seated Risks

What Is Checked. Architecture-level exposure and residual risk are checked for:

- Leftover backdoors and debug options
- Effective isolation between network segments/VLANs (lateral-movement risk)
- Exposure of internal-only services or cloud metadata endpoints (SSRF)
- Supply-chain and outdated-component risk
- Insecure coding practices, reviewed via code review

Framework Coverage. Partially automated. Nuclei and ZAP detect exposed debug/backdoor endpoints and misconfigurations; architecture and code review are performed manually and are not part of the automated scan.

Findings Overview

Severity	Source	Environment	Status	Title	Discovered
high	zap	sit	verified	Cross Site Scripting (Reflected)	2026-08-13

high	pentagi	uat	verified	UAT /engine/ API authentication gap (merged C1+CA4)	2026-08-13
high	pentagi	sit	verified	CSWSH /live/ws/locks (merged C5+CA3)	2026-08-13
high	pentagi	sit	verified	Unauthenticated FastAPI OpenAPI spec	2026-08-13
medium	zap	sit	verified	Missing X-Frame-Options	2026-08-13
medium	pentagi	sit	verified	Unauthenticated Swagger UI /engine/docs (200, 1,023 B, swagger-ui-dist@5).	2026-08-13
medium	pentagi	sit	verified	Unauthenticated ReDoc UI /engine/redoc (200, 905 B).	2026-08-13
medium	pentagi	sit	verified	Wildcard CORS on engine service (Starlette CORSMiddleware)	2026-08-13
medium	zap	uat	verified	Content Security Policy (CSP) Header Not Set	2026-08-14
medium	zap	uat	verified	Missing Anti-clickjacking Header	2026-08-14
medium	zap	sit	verified	Content Security Policy (CSP) Header Not Set	2026-08-14
medium	zap	sit	verified	Missing Anti-clickjacking Header	2026-08-14
medium	zap	uat	confirmed	CSP: Wildcard Directive	2026-08-16
medium	zap	uat	confirmed	CSP: style-src unsafe-inline	2026-08-16
medium	zap	uat	confirmed	Content Security Policy (CSP) Header Not Set	2026-08-16
medium	zap	sit	confirmed	CSP: Wildcard Directive	2026-08-16
medium	zap	sit	confirmed	CSP: style-src unsafe-inline	2026-08-16
medium	zap	sit	confirmed	Content Security Policy (CSP) Header Not Set	2026-08-16
low	nuclei	uat	confirmed	Missing Security Headers	2026-08-13
low	pentagi	sit	confirmed	JS-bundle information disclosure in SIT+UAT Altair assets (infra leak + dev access-code gate)	2026-08-13
low	pentagi	sit	confirmed	Missing security headers on non-/live/ responses (merged N1+CA1)	2026-08-13
low	zap	uat	confirmed	Private IP Disclosure	2026-08-14
low	zap	uat	confirmed	Strict-Transport-Security Header Not Set	2026-08-14
low	zap	uat	confirmed	X-Content-Type-Options Header Missing	2026-08-14
low	zap	sit	confirmed	Private IP Disclosure	2026-08-14
low	zap	sit	confirmed	Strict-Transport-Security Header Not Set	2026-08-14
low	zap	sit	confirmed	X-Content-Type-Options Header Missing	2026-08-14
low	zap	uat	confirmed	Private IP Disclosure	2026-08-16
low	zap	uat	confirmed	X-Content-Type-Options Header Missing	2026-08-16
low	zap	sit	confirmed	X-Content-Type-Options Header Missing	2026-08-16
low	zap	uat	new	Private IP Disclosure	2026-08-23
low	zap	uat	new	Strict-Transport-Security Multiple Header Entries (Non-compliant with Spec)	2026-08-23
low	zap	uat	new	X-Content-Type-Options Header Missing	2026-08-23
low	zap	sit	new	Private IP Disclosure	2026-08-23
low	zap	sit	new	Strict-Transport-Security Multiple Header Entries (Non-compliant with Spec)	2026-08-23
low	zap	sit	new	X-Content-Type-Options Header Missing	2026-08-23
info	nmap	sit	confirmed	Open port 22/tcp (ssh)	2026-08-13

info	pentagi	sit	confirmed	/live/ missing CSP, COEP, Permissions-Policy despite @fastify/helmet defaults.	2026-08-13
info	zap	sit	confirmed	Re-examine Cache-control Directives	2026-08-14
info	nmap	sit	confirmed	Open port 22/tcp (ssh)	2026-08-14
info	zap	sit	confirmed	Information Disclosure - Suspicious Comments	2026-08-16
info	zap	uat	new	Information Disclosure - Suspicious Comments	2026-08-23
info	zap	sit	new	Information Disclosure - Suspicious Comments	2026-08-23

Detailed Findings

high **Cross Site Scripting (Reflected)** #sec-3ade3390cace96c9

zap · sit · <https://crew-f8-usva-sit.roiscloud.com/live/?q=test> · status: verified · discovered: 2026-08-13

CWE: CWE-79 · **OWASP 2021:** A03 · **PCI DSS:** 6.5.7

Cross-site scripting (XSS) is an attack technique that involves echoing attacker-supplied code into a user's browsing context.

Detailed Description:

The application reflects user-controlled input (query parameters, path segments, or form fields) directly into the HTML response without sanitization or context-aware encoding. An attacker crafts a link such as `https://<host>/live/?q=<script>alert(document.cookie)</script>` and tricks a victim into opening it; the browser then executes the injected script in the application origin, enabling session-cookie theft or privileged actions performed as the victim.

Request

GET /live/?q=test HTTP/1.1

Response

HTTP/1.1 200 OK

Solution:

Never reflect unvalidated input. Apply context-aware output encoding for every insertion point (HTML element, attribute, JavaScript, URL), validate and allowlist input against a strict schema, and set a Content-Security-Policy that disallows inline scripts. Prefer frameworks that auto-escape by default.

Example: Replace an echoing 404 handler with a static response: `reply.code(404).send({ code: 404, data: null, message: 'Not Found' });` in a template, render user data with the framework's auto-escaping syntax (`{text}` in React, `{{ value }}` in most template engines) rather than raw string concatenation.

Remediation applied: Replace the reflective 404 error handler with a static response that does not echo the request URL or query string. Return a fixed JSON body and set Content-Type: application/json so reflected input cannot be interpreted as HTML/JavaScript.

[136a5aef] [sec-3ade3390cace96c9] high: Cross Site Scripting (Reflected)

```
+ await server.register(notFoundPlugin)
+ export default fp(async function notFoundPlugin(fastify: FastifyInstance) {
+   fastify.setNotFoundHandler((_request, reply) => {
+     reply.code(404).send({ code: 404, data: null, message: 'Not Found' })
```

Commit: 136a5aef

Retest: passed on 2026-08-16

Issue: <https://github.com/yuanzhu-ai/rois-ai/issues/11>

high UAT /engine/ API authentication gap (merged C1+CA4) #sec-c0ef57bdbf35c2bd

pentagi · uat · <https://crew-f8-usva-uat.roiscloud.com/engine/api/system/info> · status: verified · discovered: 2026-08-13

UAT /engine/api/system/info → 200 {"service":"ROIS Optimizer Server",...} + access-control-allow-origin: * vs SIT same endpoint → 401
www-authenticate: Bearer {"detail":"Invalid authentication credentials"}; affected <https://crew-f8-usva-uat.roiscloud.com/engine/api/system/info> (and /api/optimizers, /api/tasks/running, /api/tasks/all, /api/optimize/status|result|input|progress/{task_id}).

Detailed Description:

The engine API is reachable without authentication, so any party that can reach the service can invoke scheduling-engine operations: reading configuration and model data, triggering optimization runs, or altering state. Because the service is exposed on the UAT domain, the gap is reachable externally rather than limited to the trusted network.

Solution:

Enforce authentication on every engine endpoint, validate a token or session on each request, disable auto-generated documentation until access is controlled, restrict the service to an internal allowlist, and log and monitor access for anomalies.

Example: Require a bearer token or a reverse-proxy authentication layer on all /engine/* routes and set openapi_url=None, docs_url=None, redoc_url=None until the surface is locked down.

Remediation applied: Enforce authentication on all engine APIs by disabling the auto-generated OpenAPI, ReDoc and Swagger endpoints and removing the wildcard CORS origin. Require a valid token on every route and allowlist expected origins only.

[e40e8111] [sec-c0ef57bdbf35c2bd] high: UAT /engine/ API authentication gap (merged C1+CA4)

```
+ docs_url: null
+ redoc_url: null
+ openapi_url: null
cors:
- allow_origins:
- - "*"
+ allow_origins: []
allow_methods:
app.mount("/metrics", metrics_app)
- _cors_config = config_manager.get_config().server.cors
app.add_middleware(
    CORSMiddleware,
    allow_credentials=False, # allow_origins=["*"] False
)
```

Commit: e40e8111

Retest: passed on 2026-08-16

Issue: <https://github.com/yuanzhu-ai/rois-ai/issues/12>

pentagi · sit · wss://crew-f8-usva-sit.roiscloud.com/live/ws/locks · status: verified · discovered: 2026-08-13

wss://crew-f8-usva-sit.roiscloud.com/live/ws/locks and wss://crew-f8-usva-uat.roiscloud.com/live/ws/locks accept 101 with foreign Origin https://evil.example, no cookies/auth/frames; Sec-WebSocket-Accept s3pPLMBiTxaQ9kYGzzhZRbK+xOo=.

Detailed Description:

The WebSocket endpoint accepts handshakes from any origin. Browsers do not enforce the same-origin policy for WebSocket connections, so a malicious page the victim visits can open a connection to the same service and inherit the victim's session (Cross-Site WebSocket Hijacking), pushing or reading lock state on the victim's behalf.

Solution:

Validate the Origin header on the WebSocket handshake against an allowlist of trusted origins and reject everything else, bind the channel to an authenticated session, and re-verify the user after the handshake since the browser does not authenticate the WebSocket upgrade.

Example: if (!isAllowedWsOrigin(request.headers.origin)) reply.code(403).send({ error: "forbidden origin" }).

Remediation applied: Validate the Origin header on the WebSocket handshake for //live/ws/locks and reject connections from unexpected origins, so a malicious site cannot open a cross-site WebSocket to the locks channel.

【22004ac4】 [sec-be2d77b2278d5be9] high: CSWSH //live/ws/locks (merged C5+CA3)

```
+ export const isAllowedWsOrigin = (origin: string | undefined, host: string | undefined): boolean => {  
+   if (!origin) return true  
+   try {  
+     const originUrl = new URL(origin)
```

Commit: 22004ac4

Retest: passed on 2026-08-16

Issue: <https://github.com/yuanzhu-ai/rois-ai/issues/13>

high Unauthenticated FastAPI OpenAPI spec #sec-9023d758cf2ab0b6

pentagi · sit · /engine/openapi.json · status: verified · discovered: 2026-08-13

SIT+UAT /engine/openapi.json → 200, 18,096 B, OpenAPI 3.1.0, 16 paths incl. destructive POST/DELETE/PUT routes; byte-identical SIT vs UAT.

Detailed Description:

The FastAPI OpenAPI schema (openapi.json) is served without authentication, disclosing the complete API surface - every route, parameter, model and operation - to anyone who can reach the service. This metadata is a ready-made attack map for the engine API.

Solution:

Disable the schema in production with openapi_url=None, or gate it behind authentication and an internal-only network allowlist.

Example: FastAPI(..., openapi_url=None).

Remediation applied: Disable the auto-generated FastAPI OpenAPI schema by setting openapi_url to null in the FastAPI app configuration, preventing unauthenticated disclosure of the API surface.

[e40e8111] [sec-9023d758cf2ab0b6] high: Unauthenticated FastAPI OpenAPI spec

```
+ docs_url: null
+ redoc_url: null
+ openapi_url: null
  cors:
-   allow_origins:
-     - "*"
+   allow_origins: []
+   allow_methods:
+   app.mount("/metrics", metrics_app)
-   _cors_config = config_manager.get_config().server.cors
+   app.add_middleware(
+     CORSMiddleware,
+     allow_credentials=False, # allow_origins=["*"] False
+   )
```

Commit: e40e8111

Retest: passed on 2026-08-16

Issue: <https://github.com/yuanzhu-ai/rois-ai/issues/14>

medium **Missing X-Frame-Options** #sec-f838f93c10f395cc

zap · sit · <https://crew-f8-usva-sit.roiscloud.com/altair/> · status: verified · discovered: 2026-08-13

CWE: CWE-1021 · **OWASP 2021:** · **PCI DSS:**

The X-Frame-Options header is missing.

Detailed Description:

Responses do not carry X-Frame-Options or the frame-ancestors Content-Security-Policy directive, so the page can be embedded in a third-party <iframe>. An attacker overlays transparent elements or uses CSS to lure the victim into clicking buttons or submitting forms inside the framed page (clickjacking / UI redress), triggering sensitive actions such as roster or privilege changes without the user's intent.

Request

GET /altair/ HTTP/1.1

Response

HTTP/1.1 200 OK

Solution:

Send X-Frame-Options: DENY on every response, or the modern Content-Security-Policy frame-ancestors 'none' directive (CSP takes precedence when both are present). Only allow framing for domains with a genuine requirement.

Example: In nginx: add_header X-Frame-Options "DENY" always;. In Fastify/Express: helmet.frameguard({ action: 'deny' }) or app.use(helmet()) which sets it by default.

Remediation applied: Add the X-Frame-Options: DENY response header to prevent the application from being rendered inside a cross-origin frame.

【a4e686b7】 [sec-f838f93c10f395cc] medium: Missing X-Frame-Options

```
+ add_header X-Frame-Options "DENY" always;
+ add_header X-Content-Type-Options "nosniff" always;
+ add_header Referrer-Policy "strict-origin-when-cross-origin" always;
+ add_header Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data: blob:; font-src 'self' data:; connect-src 'self' ws: wss:; object-src 'none'; frame-ancestors 'none'; base-uri 'self'; form-action 'self';" always;
```

Commit: a4e686b7

Retest: passed on 2026-08-16

Issue: <https://github.com/yuanzhu-ai/rois-ai/issues/15>

medium

Unauthenticated Swagger UI /engine/docs (200, 1,023 B, swagger-ui-dist@5). #sec-57388404fa80cfd7

pentagi · sit · /engine/docs · status: verified · discovered: 2026-08-13

Unauthenticated Swagger UI /engine/docs (200, 1,023 B, swagger-ui-dist@5).

Detailed Description:

FastAPI auto-generates interactive documentation - the Swagger UI at /docs, ReDoc at /redoc and the underlying OpenAPI schema - and serves it without authentication. Anyone who can reach the service can enumerate every endpoint, parameter and data shape of the engine API, turning the schema into a roadmap for targeted attacks.

Solution:

Disable auto-generated documentation in non-development deployments (openapi_url=None, docs_url=None, redoc_url=None) or protect it behind authentication; keep the schema internal to the engineering team.

Example: FastAPI(..., openapi_url=None, docs_url=None, redoc_url=None).

Remediation applied: Disable the auto-generated Swagger UI by setting docs_url to null in the FastAPI app configuration so the interactive API documentation is not publicly reachable.

【e40e8111】 [sec-57388404fa80cfd7] medium: Unauthenticated Swagger UI /engine/docs (200, 1,023 B, swagger-ui-dist@5).

```
+ docs_url: null
+ redoc_url: null
+ openapi_url: null
cors:
- allow_origins:
- - "*"
+ allow_origins: []
allow_methods:
app.mount("/metrics", metrics_app)
- _cors_config = config_manager.get_config().server.cors
app.add_middleware(
    CORSMiddleware,
    allow_credentials=False, # allow_origins=["*"] False
)
```

Commit: e40e8111

Retest: passed on 2026-08-16

Issue: <https://github.com/yuanzhu-ai/rois-ai/issues/16>

medium

Unauthenticated ReDoc UI /engine/redoc (200, 905 B). #sec-75f507a27694b07d

pentagi · sit · /engine/redoc · status: verified · discovered: 2026-08-13

Unauthenticated ReDoc UI /engine/redoc (200, 905 B).

Detailed Description:

FastAPI auto-generates interactive documentation - the Swagger UI at /docs, ReDoc at /redoc and the underlying OpenAPI schema - and serves it without authentication. Anyone who can reach the service can enumerate every endpoint, parameter and data shape of the engine API, turning the schema into a roadmap for targeted attacks.

Solution:

Disable auto-generated documentation in non-development deployments (openapi_url=None, docs_url=None, redoc_url=None) or protect it behind authentication; keep the schema internal to the engineering team.

Example: FastAPI(..., openapi_url=None, docs_url=None, redoc_url=None).

Remediation applied: Disable the auto-generated ReDoc UI by setting redoc_url to null in the FastAPI app configuration so the interactive API documentation is not publicly reachable.

【e40e8111】 [sec-75f507a27694b07d] medium: Unauthenticated ReDoc UI /engine/redoc (200, 905 B).

```
+ docs_url: null
+ redoc_url: null
+ openapi_url: null
cors:
- allow_origins:
- - "*"
+ allow_origins: []
allow_methods:
app.mount("/metrics", metrics_app)
- _cors_config = config_manager.get_config().server.cors
app.add_middleware(
    CORSMiddleware,
    allow_credentials=False, # allow_origins=["*"] False
)
```

Commit: e40e8111

Retest: passed on 2026-08-16

Issue: <https://github.com/yuanzhu-ai/rois-ai/issues/17>

medium

Wildcard CORS on engine service (Starlette CORSMiddleware) #sec-67393692463874b2

pentagi · sit · status: verified · discovered: 2026-08-13

access-control-allow-origin: * for any Origin; preflight 200 text/plain "OK", allow-methods GET, POST, allow-headers incl. Authorization/X-API-Key/X-Airline, max-age 600.

Detailed Description:

The engine service answers cross-origin requests with Access-Control-Allow-Origin: * (the Starlette CORSMiddleware default), so any website can read responses from the engine API in a victim's browser context. If the API returns data or accepts state-changing calls, a malicious site can exfiltrate data or trigger actions cross-origin.

Solution:

Replace the wildcard with an explicit allowlist of the expected application origins, do not echo the Origin header, keep allow_credentials=False (browsers reject it combined with *), and restrict methods to the minimal set.

Example: CORSMiddleware(allow_origins=["https://crew-f8-usva-sit.roiscloud.com", "https://crew-f8-usva-uat.roiscloud.com"], allow_methods=["GET", "POST"], allow_credentials=False).

Remediation applied: Replace the wildcard CORS origin with an explicit allowlist of expected origins, or disable CORS entirely where the engine is not consumed cross-origin.

[e40e8111] [sec-67393692463874b2] medium: Wildcard CORS on engine service (Starlette CORSMiddleware)

```
+ docs_url: null
+ redoc_url: null
+ openapi_url: null
cors:
- allow_origins:
- - "*"
+ allow_origins: []
allow_methods:
app.mount("/metrics", metrics_app)
- _cors_config = config_manager.get_config().server.cors
app.add_middleware(
CORSMiddleware,
allow_credentials=False, # allow_origins=["*"] False
)
```

Commit: e40e8111

Retest: passed on 2026-08-16

Issue: <https://github.com/yuanzhu-ai/rois-ai/issues/18>

medium

Content Security Policy (CSP) Header Not Set #sec-81bb353a41ba97f1

zap · uat · <https://crew-f8-usva-uat.roiscloud.com/altair/> · status: verified · discovered: 2026-08-14

CWE: CWE-693 · **OWASP 2021:** · **PCI DSS:**

<p>Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.</p>

Detailed Description:

Responses omit a Content-Security-Policy and related hardening headers (for example X-Content-Type-Options), so the browser falls back to permissive defaults. Any injection or script-loading weakness is amplified because the browser will execute attacker-supplied scripts and styles, and without nosniff a browser may reinterpret a non-HTML response (JSON, images) as a document, turning otherwise benign content into a script-execution vector.

Solution:

Set a strict Content-Security-Policy that constrains default-src, script-src, object-src and style-src and disallows inline and eval where feasible; add X-Content-Type-Options: nosniff. Apply the headers to every response, not only selected routes, and roll out with Content-Security-Policy-Report-Only first to detect breakage.

Example: Header set Content-Security-Policy "default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none'"; Header set X-Content-Type-Options "nosniff".

Remediation applied: Add a Content Security Policy response header that restricts script, style, and frame sources to trusted origins, and verify the application still loads with the policy applied.

【a4e686b7】 [sec-81bb353a41ba97f1] medium: Content Security Policy (CSP) Header Not Set

```
+ add_header X-Frame-Options "DENY" always;
+ add_header X-Content-Type-Options "nosniff" always;
+ add_header Referrer-Policy "strict-origin-when-cross-origin" always;
+ add_header Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data: blob;; font-src 'self' data;; connect-src 'self' ws: wss;; object-src 'none'; frame-ancestors 'none'; base-uri 'self'; form-action 'self';" always;
```

Commit: a4e686b7

Retest: passed on 2026-08-16

Issue: <https://github.com/yuanzhu-ai/rois-ai/issues/19>

CWE: CWE-1021 · **OWASP 2021:** · **PCI DSS:**

<p>The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.</p>

Detailed Description:

Responses do not carry X-Frame-Options or the frame-ancestors Content-Security-Policy directive, so the page can be embedded in a third-party <iframe>. An attacker overlays transparent elements or uses CSS to lure the victim into clicking buttons or submitting forms inside the framed page (clickjacking / UI redress), triggering sensitive actions such as roster or privilege changes without the user's intent.

Solution:

Send X-Frame-Options: DENY on every response, or the modern Content-Security-Policy frame-ancestors 'none' directive (CSP takes precedence when both are present). Only allow framing for domains with a genuine requirement.

Example: In nginx: add_header X-Frame-Options "DENY" always;. In Fastify/Express: helmet.frameguard({ action: 'deny' }) or app.use(helmet()) which sets it by default.

Remediation applied: Add the X-Frame-Options: DENY response header (or the frame-ancestors Content-Security-Policy directive) to prevent clickjacking and UI redressing.

【a4e686b7】 [sec-cc3eaa987ac95c7e] medium: Missing Anti-clickjacking Header

```
+ add_header X-Frame-Options "DENY" always;
+ add_header X-Content-Type-Options "nosniff" always;
+ add_header Referrer-Policy "strict-origin-when-cross-origin" always;
+ add_header Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data: blob;; font-src 'self' data;; connect-src 'self' ws: wss;; object-src 'none'; frame-ancestors 'none'; base-uri 'self'; form-action 'self';" always;
```

Commit: a4e686b7

Retest: passed on 2026-08-16

Issue: <https://github.com/yuanzhu-ai/rois-ai/issues/20>

zap · sit · <https://crew-f8-usva-sit.roiscloud.com/altair/> · status: verified · discovered: 2026-08-14

CWE: CWE-693 · **OWASP 2021:** · **PCI DSS:**

<p>Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.</p>

Detailed Description:

Responses omit a Content-Security-Policy and related hardening headers (for example X-Content-Type-Options), so the browser falls back to permissive defaults. Any injection or script-loading weakness is amplified because the browser will execute attacker-supplied scripts and styles, and without nosniff a browser may reinterpret a non-HTML response (JSON, images) as a document, turning otherwise benign content into a script-execution vector.

Solution:

Set a strict Content-Security-Policy that constrains default-src, script-src, object-src and style-src and disallows inline and eval where feasible; add X-Content-Type-Options: nosniff. Apply the headers to every response, not only selected routes, and roll out with Content-Security-Policy-Report-Only first to detect breakage.

Example: Header set Content-Security-Policy "default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none'"; Header set X-Content-Type-Options "nosniff".

Remediation applied: Add a Content Security Policy response header that restricts script, style, and frame sources to trusted origins, and verify the application still loads with the policy applied.

【a4e686b7】 [sec-e876bb78459508c4] medium: Content Security Policy (CSP) Header Not Set

```
+ add_header X-Frame-Options "DENY" always;
+ add_header X-Content-Type-Options "nosniff" always;
+ add_header Referrer-Policy "strict-origin-when-cross-origin" always;
+ add_header Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data: blob;; font-src 'self' data;; connect-src 'self' ws: wss;; object-src 'none'; frame-ancestors 'none'; base-uri 'self'; form-action 'self';" always;
```

Commit: a4e686b7

Retest: passed on 2026-08-16

Issue: <https://github.com/yuanzhu-ai/rois-ai/issues/21>

medium

Missing Anti-clickjacking Header #sec-3930dc7acec994a1

zap · sit · <https://crew-f8-usva-sit.roiscloud.com/altair/> · status: verified · discovered: 2026-08-14

CWE: CWE-1021 · **OWASP 2021:** · **PCI DSS:**

<p>The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.</p>

Detailed Description:

Responses do not carry X-Frame-Options or the frame-ancestors Content-Security-Policy directive, so the page can be embedded in a third-party <iframe>. An attacker overlays transparent elements or uses CSS to lure the victim into clicking buttons or submitting forms inside the framed page (clickjacking / UI redress), triggering sensitive actions such as roster or privilege changes without the user's intent.

Solution:

Send X-Frame-Options: DENY on every response, or the modern Content-Security-Policy frame-ancestors 'none' directive (CSP takes precedence when both are present). Only allow framing for domains with a genuine requirement.

Example: In nginx: add_header X-Frame-Options "DENY" always;. In Fastify/Express: helmet.frameguard({ action: 'deny' }) or app.use(helmet()) which sets it by default.

Remediation applied: Add the X-Frame-Options: DENY response header (or the frame-ancestors Content-Security-Policy directive) to prevent clickjacking and UI redressing.

【a4e686b7】 [sec-3930dc7acec994a1] medium: Missing Anti-clickjacking Header

```
+ add_header X-Frame-Options "DENY" always;
+ add_header X-Content-Type-Options "nosniff" always;
+ add_header Referrer-Policy "strict-origin-when-cross-origin" always;
+ add_header Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data: blob;; font-src 'self' data;; connect-src 'self' ws: wss;; object-src 'none'; frame-ancestors 'none'; base-uri 'self'; form-action 'self';" always;
```

Commit: a4e686b7

Retest: passed on 2026-08-16

Issue: <https://github.com/yuanzhu-ai/rois-ai/issues/22>

medium **CSP: Wildcard Directive** #sec-9f9b54de48cf6430

zap · uat · <https://crew-f8-usva-uat.roiscloud.com/altair/> · status: confirmed · discovered: 2026-08-16

CWE: CWE-693 · **OWASP 2021:** · **PCI DSS:**

<p>Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks. Including (but not limited to) Cross Site Scripting (XSS), and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.</p>

Detailed Description:

Responses omit a Content-Security-Policy and related hardening headers (for example X-Content-Type-Options), so the browser falls back to permissive defaults. Any injection or script-loading weakness is amplified because the browser will execute attacker-supplied scripts and styles, and without nosniff a browser may reinterpret a non-HTML response (JSON, images) as a document, turning otherwise benign content into a script-execution vector.

Solution:

Set a strict Content-Security-Policy that constrains default-src, script-src, object-src and style-src and disallows inline and eval where feasible; add X-Content-Type-Options: nosniff. Apply the headers to every response, not only selected routes, and roll out with Content-Security-Policy-Report-Only first to detect breakage.

Example: Header set Content-Security-Policy "default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none'"; Header set X-Content-Type-Options "nosniff".

medium **CSP: style-src unsafe-inline** #sec-0a6564308d6c6caa

zap · uat · <https://crew-f8-usva-uat.roiscloud.com/altair/> · status: confirmed · discovered: 2026-08-16

CWE: CWE-693 · **OWASP 2021:** · **PCI DSS:**

<p>Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks. Including (but not limited to) Cross Site Scripting (XSS), and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.</p>

Detailed Description:

Responses omit a Content-Security-Policy and related hardening headers (for example X-Content-Type-Options), so the browser falls back to permissive defaults. Any injection or script-loading weakness is amplified because the browser will execute attacker-supplied scripts and styles, and without nosniff a browser may reinterpret a non-HTML response (JSON, images) as a document, turning otherwise benign content into a script-execution vector.

Solution:

Set a strict Content-Security-Policy that constrains default-src, script-src, object-src and style-src and disallows inline and eval where feasible; add X-Content-Type-Options: nosniff. Apply the headers to every response, not only selected routes, and roll out with Content-Security-Policy-Report-Only first to detect breakage.

Example: Header set Content-Security-Policy "default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none'"; Header set X-Content-Type-Options "nosniff".

medium **Content Security Policy (CSP) Header Not Set** #sec-e09ffd1720f008d1

zap · uat · <https://crew-f8-usva-uat.roiscloud.com/rule/> · status: confirmed · discovered: 2026-08-16

CWE: CWE-693 · **OWASP 2021:** · **PCI DSS:**

<p>Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.</p>

Detailed Description:

Responses omit a Content-Security-Policy and related hardening headers (for example X-Content-Type-Options), so the browser falls back to permissive defaults. Any injection or script-loading weakness is amplified because the browser will execute attacker-supplied scripts and styles, and without nosniff a browser may reinterpret a non-HTML response (JSON, images) as a document, turning otherwise benign content into a script-execution vector.

Solution:

Set a strict Content-Security-Policy that constrains default-src, script-src, object-src and style-src and disallows inline and eval where feasible; add X-Content-Type-Options: nosniff. Apply the headers to every response, not only selected routes, and roll out with Content-Security-Policy-Report-Only first to detect breakage.

Example: Header set Content-Security-Policy "default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none'"; Header set X-Content-Type-Options "nosniff".

medium **CSP: Wildcard Directive** #sec-6e248ba3272e2dd9

zap · sit · <https://crew-f8-usva-sit.roiscloud.com/altair/> · status: confirmed · discovered: 2026-08-16

CWE: CWE-693 · **OWASP 2021:** · **PCI DSS:**

<p>Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks. Including (but not limited to) Cross Site Scripting (XSS), and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.</p>

Detailed Description:

Responses omit a Content-Security-Policy and related hardening headers (for example X-Content-Type-Options), so the browser falls back to permissive defaults. Any injection or script-loading weakness is amplified because the browser will execute attacker-supplied scripts and styles, and without nosniff a browser may reinterpret a non-HTML response (JSON, images) as a document, turning otherwise benign content into a script-execution vector.

Solution:

Set a strict Content-Security-Policy that constrains default-src, script-src, object-src and style-src and disallows inline and eval where feasible; add X-Content-Type-Options: nosniff. Apply the headers to every response, not only selected routes, and roll out with Content-Security-Policy-Report-Only first to detect breakage.

Example: Header set Content-Security-Policy "default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none'"; Header set X-Content-Type-Options "nosniff".

medium

CSP: style-src unsafe-inline #sec-5532716f636226d7

zap · sit · <https://crew-f8-usva-sit.roiscloud.com/altair/> · status: confirmed · discovered: 2026-08-16

CWE: CWE-693 · **OWASP 2021:** · **PCI DSS:**

<p>Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks. Including (but not limited to) Cross Site Scripting (XSS), and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.</p>

Detailed Description:

Responses omit a Content-Security-Policy and related hardening headers (for example X-Content-Type-Options), so the browser falls back to permissive defaults. Any injection or script-loading weakness is amplified because the browser will execute attacker-supplied scripts and styles, and without nosniff a browser may reinterpret a non-HTML response (JSON, images) as a document, turning otherwise benign content into a script-execution vector.

Solution:

Set a strict Content-Security-Policy that constrains default-src, script-src, object-src and style-src and disallows inline and eval where feasible; add X-Content-Type-Options: nosniff. Apply the headers to every response, not only selected routes, and roll out with Content-Security-Policy-Report-Only first to detect breakage.

Example: Header set Content-Security-Policy "default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none'"; Header set X-Content-Type-Options "nosniff".

medium

Content Security Policy (CSP) Header Not Set #sec-75cb61c1857ac72d

zap · sit · <https://crew-f8-usva-sit.roiscloud.com/rule/> · status: confirmed · discovered: 2026-08-16

CWE: CWE-693 · **OWASP 2021:** · **PCI DSS:**

<p>Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.</p>

Detailed Description:

Responses omit a Content-Security-Policy and related hardening headers (for example X-Content-Type-Options), so the browser falls back to permissive defaults. Any injection or script-loading weakness is amplified because the browser will execute attacker-supplied scripts and styles, and without nosniff a browser may reinterpret a non-HTML response (JSON, images) as a document, turning otherwise benign content into a script-execution vector.

Solution:

Set a strict Content-Security-Policy that constrains default-src, script-src, object-src and style-src and disallows inline and eval where feasible; add X-Content-Type-Options: nosniff. Apply the headers to every response, not only selected routes, and roll out with Content-Security-Policy-Report-Only first to detect breakage.

Example: Header set Content-Security-Policy "default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none'"; Header set X-Content-Type-Options "nosniff".

low Missing Security Headers #sec-1f33337710d24de2

nuclei · uat · <https://crew-f8-usva-uat.roiscloud.com/altair/> · status: confirmed · discovered: 2026-08-13

Detected by template http-missing-security-headers

Detailed Description:

Responses omit recommended hardening headers (HSTS, X-Frame-Options, CSP, X-Content-Type-Options, Referrer-Policy, Permissions-Policy). Each missing header is low risk on its own, but together they remove the defense-in-depth layers that would contain injection, clickjacking and protocol-downgrade attacks.

Response

HTTP/1.1 200 OK

Solution:

Apply a consistent header baseline across every response at the web-server layer (HSTS, frameguard, CSP, nosniff, Referrer-Policy, Permissions-Policy) and align with the platform hardening standard.

Example: A single nginx add_header block (with always) setting X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy and Strict-Transport-Security.

low JS-bundle information disclosure in SIT+UAT Altair assets (infra leak + dev access-code gate) #sec-

6659f374e6192348

pentagi · sit · <https://crew-f8-usva-sit.roiscloud.com/altair/assets/index-tVbqGDXM.js> · status: confirmed · discovered: 2026-08-13

registered against the in-scope asset URLs <https://crew-f8-usva-sit.roiscloud.com/altair/assets/index-tVbqGDXM.js> and <https://crew-f8-usva-uat.roiscloud.com/altair/assets/index-CiXcTYYo.js>.

Detailed Description:

The shipped JavaScript bundles for the Altair front-end embed runtime configuration - internal API endpoints, the dev access-code gate and environment details - which anyone can read by viewing the bundle source. This exposes internal infrastructure paths and the mechanism behind a developer-only gate, reducing the effort required to reach internal services.

Solution:

Remove internal endpoints, secrets and gate codes from client bundles; serve runtime configuration from a server-provided endpoint behind access control; keep environment-specific data out of publicly delivered JavaScript.

Example: Fetch runtime configuration from an authenticated /api/config endpoint instead of embedding it in the bundle at build time.

low Missing security headers on non-/live/ responses (merged N1+CA1) #sec-7ab6e26d2bdf84ec

pentagi · sit · /live/ · status: confirmed · discovered: 2026-08-13

CWE: CWE-693 · **OWASP 2021:** · **PCI DSS:**

all 11 non-/live/ URLs; all 10 standard headers absent; nuclei 116 info matches CWE-693.

Detailed Description:

Responses omit a Content-Security-Policy and related hardening headers (for example X-Content-Type-Options), so the browser falls back to permissive defaults. Any injection or script-loading weakness is amplified because the browser will execute attacker-supplied scripts and styles, and without nosniff a browser may reinterpret a non-HTML response (JSON, images) as a document, turning otherwise benign content into a script-execution vector.

Solution:

Set a strict Content-Security-Policy that constrains default-src, script-src, object-src and style-src and disallows inline and eval where feasible; add X-Content-Type-Options: nosniff. Apply the headers to every response, not only selected routes, and roll out with Content-Security-Policy-Report-Only first to detect breakage.

Example: Header set Content-Security-Policy "default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none'"; Header set X-Content-Type-Options "nosniff".

low Private IP Disclosure #sec-ebc3826cc61777c0

zap · uat · https://crew-f8-usva-uat.roiscloud.com/altair/assets/index-DbYzIMxG.js · status: confirmed · discovered: 2026-08-14

CWE: CWE-497 · **OWASP 2021:** · **PCI DSS:**

<p>A private IP (such as 10.x.x.x, 172.x.x.x, 192.168.x.x) or an Amazon EC2 private hostname (for example, ip-10-0-56-78) has been found in the HTTP response body. This information might be helpful for further attacks targeting internal systems.</p>

Detailed Description:

The application discloses internal network addresses (RFC 1918 private IPs such as 10.x.x.x or 192.168.x.x) in responses, error messages, stack traces, or page source. An attacker uses the leaked addresses to map the internal network, infer infrastructure topology, and target internal services that would otherwise be unreachable or firewalled.

Solution:

Remove internal addresses and hostnames from all responses, error pages, stack traces and logs; return generic error identifiers; mask internal addresses behind the public hostname; ensure load balancer and monitoring headers do not leak backend IPs.

Example: Return { "error": "internal_error", "ref": "a1b2c3" } instead of a stack trace containing 10.0.4.12; strip backend IP headers at the proxy.

low Strict-Transport-Security Header Not Set #sec-74c07cb514667aad

zap · uat · <https://crew-f8-usva-uat.roiscloud.com/altair/> · status: confirmed · discovered: 2026-08-14

CWE: CWE-319 · **OWASP 2021:** · **PCI DSS:**

<p>HTTP Strict Transport Security (HSTS) is a web security policy mechanism whereby a web server declares that complying user agents (such as a web browser) are to interact with it using only secure HTTPS connections (i.e. HTTP layered over TLS/SSL). HSTS is an IETF standards track protocol and is specified in RFC 6797.</p>

Detailed Description:

Responses do not include the Strict-Transport-Security (HSTS) header, so a client that first reaches the site over HTTP - or falls back to cleartext within a mixed-content context - can be downgraded or stripped by an on-path attacker. Without HSTS the browser neither forces HTTPS on later visits nor refuses invalid certificates, leaving session credentials and data exposed to interception.

Solution:

Serve all traffic over TLS and send Strict-Transport-Security from every HTTPS response (max-age of at least six months, includeSubDomains, and preload once subdomains are verified). Redirect all HTTP requests to HTTPS and disable weak protocols and cipher suites.

Example: Strict-Transport-Security "max-age=63072000; includeSubDomains; preload" alongside a 301 HTTP-to-HTTPS redirect.

low X-Content-Type-Options Header Missing #sec-8bab699d073eea6b

zap · uat · <https://crew-f8-usva-uat.roiscloud.com/altair/> · status: confirmed · discovered: 2026-08-14

CWE: CWE-693 · **OWASP 2021:** · **PCI DSS:**

<p>The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.</p>

Detailed Description:

Responses omit a Content-Security-Policy and related hardening headers (for example X-Content-Type-Options), so the browser falls back to permissive defaults. Any injection or script-loading weakness is amplified because the browser will execute attacker-supplied scripts and styles, and without nosniff a browser may reinterpret a non-HTML response (JSON, images) as a document, turning otherwise benign content into a script-execution vector.

Solution:

Set a strict Content-Security-Policy that constrains default-src, script-src, object-src and style-src and disallows inline and eval where feasible; add X-Content-Type-Options: nosniff. Apply the headers to every response, not only selected routes, and roll out with Content-Security-Policy-Report-Only first to detect breakage.

Example: Header set Content-Security-Policy "default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none'"; Header set X-Content-Type-Options "nosniff".

low Private IP Disclosure #sec-e58cc756305f4a03

zap · sit · <https://crew-f8-usva-sit.roiscloud.com/altair/assets/index-Dt4wplig.js> · status: confirmed · discovered: 2026-08-14

CWE: CWE-497 · **OWASP 2021:** · **PCI DSS:**

<p>A private IP (such as 10.x.x.x, 172.x.x.x, 192.168.x.x) or an Amazon EC2 private hostname (for example, ip-10-0-56-78) has been found in the HTTP response body. This information might be helpful for further attacks targeting internal systems.</p>

Detailed Description:

The application discloses internal network addresses (RFC 1918 private IPs such as 10.x.x.x or 192.168.x.x) in responses, error messages, stack traces, or page source. An attacker uses the leaked addresses to map the internal network, infer infrastructure topology, and target internal services that would otherwise be unreachable or firewalled.

Solution:

Remove internal addresses and hostnames from all responses, error pages, stack traces and logs; return generic error identifiers; mask internal addresses behind the public hostname; ensure load balancer and monitoring headers do not leak backend IPs.

Example: Return { "error": "internal_error", "ref": "a1b2c3" } instead of a stack trace containing 10.0.4.12; strip backend IP headers at the proxy.

low Strict-Transport-Security Header Not Set #sec-686828c1de7530db

zap · sit · <https://crew-f8-usva-sit.roiscloud.com/altair/> · status: confirmed · discovered: 2026-08-14

CWE: CWE-319 · **OWASP 2021:** · **PCI DSS:**

<p>HTTP Strict Transport Security (HSTS) is a web security policy mechanism whereby a web server declares that complying user agents (such as a web browser) are to interact with it using only secure HTTPS connections (i.e. HTTP layered over TLS/SSL). HSTS is an IETF standards track protocol and is specified in RFC 6797.</p>

Detailed Description:

Responses do not include the Strict-Transport-Security (HSTS) header, so a client that first reaches the site over HTTP - or falls back to cleartext within a mixed-content context - can be downgraded or stripped by an on-path attacker. Without HSTS the browser neither forces HTTPS on later visits nor refuses invalid certificates, leaving session credentials and data exposed to interception.

Solution:

Serve all traffic over TLS and send Strict-Transport-Security from every HTTPS response (max-age of at least six months, includeSubDomains, and preload once subdomains are verified). Redirect all HTTP requests to HTTPS and disable weak protocols and cipher suites.

Example: Strict-Transport-Security "max-age=63072000; includeSubDomains; preload" alongside a 301 HTTP-to-HTTPS redirect.

low X-Content-Type-Options Header Missing #sec-2a58b70dd36fd4ff

zap · sit · <https://crew-f8-usva-sit.roiscloud.com/altair/> · status: confirmed · discovered: 2026-08-14

CWE: CWE-693 · **OWASP 2021:** · **PCI DSS:**

<p>The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.</p>

Detailed Description:

Responses omit a Content-Security-Policy and related hardening headers (for example X-Content-Type-Options), so the browser falls back to permissive defaults. Any injection or script-loading weakness is amplified because the browser will execute attacker-supplied scripts and styles, and without nosniff a browser may reinterpret a non-HTML response (JSON, images) as a document, turning otherwise benign content into a script-execution vector.

Solution:

Set a strict Content-Security-Policy that constrains default-src, script-src, object-src and style-src and disallows inline and eval where feasible; add X-Content-Type-Options: nosniff. Apply the headers to every response, not only selected routes, and roll out with Content-Security-Policy-Report-Only first to detect breakage.

Example: Header set Content-Security-Policy "default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none'"; Header set X-Content-Type-Options "nosniff".

low Private IP Disclosure #sec-c78d1b80295afd24

zap · uat · <https://crew-f8-usva-uat.roiscloud.com/altair/assets/index-BiG1o77t.js> · status: confirmed · discovered: 2026-08-16

CWE: CWE-497 · **OWASP 2021:** · **PCI DSS:**

<p>A private IP (such as 10.x.x.x, 172.x.x.x, 192.168.x.x) or an Amazon EC2 private hostname (for example, ip-10-0-56-78) has been found in the HTTP response body. This information might be helpful for further attacks targeting internal systems.</p>

Detailed Description:

The application discloses internal network addresses (RFC 1918 private IPs such as 10.x.x.x or 192.168.x.x) in responses, error messages, stack traces, or page source. An attacker uses the leaked addresses to map the internal network, infer infrastructure topology, and target internal services that would otherwise be unreachable or firewalled.

Solution:

Remove internal addresses and hostnames from all responses, error pages, stack traces and logs; return generic error identifiers; mask internal addresses behind the public hostname; ensure load balancer and monitoring headers do not leak backend IPs.

Example: Return { "error": "internal_error", "ref": "a1b2c3" } instead of a stack trace containing 10.0.4.12; strip backend IP headers at the proxy.

low X-Content-Type-Options Header Missing #sec-1e3eb1423913606a

zap · uat · <https://crew-f8-usva-uat.roiscloud.com/altair/assets/index-CV8x76kH.css> · status: confirmed · discovered: 2026-08-16

CWE: CWE-693 · **OWASP 2021:** · **PCI DSS:**

<p>The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.</p>

Detailed Description:

Responses omit a Content-Security-Policy and related hardening headers (for example X-Content-Type-Options), so the browser falls back to permissive defaults. Any injection or script-loading weakness is amplified because the browser will execute attacker-supplied scripts and styles, and without nosniff a browser may reinterpret a non-HTML response (JSON, images) as a document, turning otherwise benign content into a script-execution vector.

Solution:

Set a strict Content-Security-Policy that constrains default-src, script-src, object-src and style-src and disallows inline and eval where feasible; add X-Content-Type-Options: nosniff. Apply the headers to every response, not only selected routes, and roll out with Content-Security-Policy-Report-Only first to detect breakage.

Example: Header set Content-Security-Policy "default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none'"; Header set X-Content-Type-Options "nosniff".

low X-Content-Type-Options Header Missing #sec-5394c8d6234b38ac

zap · sit · <https://crew-f8-usva-sit.roiscloud.com/altair/assets/index-CfYvi3UN.css> · status: confirmed · discovered: 2026-08-16

CWE: CWE-693 · **OWASP 2021:** · **PCI DSS:**

<p>The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.</p>

Detailed Description:

Responses omit a Content-Security-Policy and related hardening headers (for example X-Content-Type-Options), so the browser falls back to permissive defaults. Any injection or script-loading weakness is amplified because the browser will execute attacker-supplied scripts and styles, and without nosniff a browser may reinterpret a non-HTML response (JSON, images) as a document, turning otherwise benign content into a script-execution vector.

Solution:

Set a strict Content-Security-Policy that constrains default-src, script-src, object-src and style-src and disallows inline and eval where feasible; add X-Content-Type-Options: nosniff. Apply the headers to every response, not only selected routes, and roll out with Content-Security-Policy-Report-Only first to detect breakage.

Example: Header set Content-Security-Policy "default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none'"; Header set X-Content-Type-Options "nosniff".

low Private IP Disclosure [#sec-a9347a44f3ef2dfe](#)

zap · uat · <https://crew-f8-usva-uat.roiscloud.com/altair/assets/index-B0tMGYa5.js> · status: new · discovered: 2026-08-23

CWE: CWE-497 · **OWASP 2021:** · **PCI DSS:**

<p>A private IP (such as 10.x.x.x, 172.x.x.x, 192.168.x.x) or an Amazon EC2 private hostname (for example, ip-10-0-56-78) has been found in the HTTP response body. This information might be helpful for further attacks targeting internal systems.</p>

Detailed Description:

The application discloses internal network addresses (RFC 1918 private IPs such as 10.x.x.x or 192.168.x.x) in responses, error messages, stack traces, or page source. An attacker uses the leaked addresses to map the internal network, infer infrastructure topology, and target internal services that would otherwise be unreachable or firewalled.

Solution:

Remove internal addresses and hostnames from all responses, error pages, stack traces and logs; return generic error identifiers; mask internal addresses behind the public hostname; ensure load balancer and monitoring headers do not leak backend IPs.

Example: Return { "error": "internal_error", "ref": "a1b2c3" } instead of a stack trace containing 10.0.4.12; strip backend IP headers at the proxy.

low Strict-Transport-Security Multiple Header Entries (Non-compliant with Spec) [#sec-dfaae6c451e24e0b](#)

zap · uat · <https://crew-f8-usva-uat.roiscloud.com/live/> · status: new · discovered: 2026-08-23

CWE: CWE-319 · **OWASP 2021:** · **PCI DSS:**

<p>HTTP Strict Transport Security (HSTS) headers were found, a response with multiple HSTS header entries is not compliant with the specification (RFC 6797) and only the first HSTS header will be processed others will be ignored by user agents or the HSTS policy may be incorrectly applied.</p><p>HTTP Strict Transport Security (HSTS) is a web security policy mechanism whereby a web server declares that complying user agents (such as a web browser) are to interact with it using only secure HTTPS connections (i.e. HTTP layered over TLS/SSL).</p>

Detailed Description:

Responses do not include the Strict-Transport-Security (HSTS) header, so a client that first reaches the site over HTTP - or falls back to cleartext within a mixed-content context - can be downgraded or stripped by an on-path attacker. Without HSTS the browser neither forces HTTPS on later visits nor refuses invalid certificates, leaving session credentials and data exposed to interception.

Solution:

Serve all traffic over TLS and send Strict-Transport-Security from every HTTPS response (max-age of at least six months, includeSubDomains, and preload once subdomains are verified). Redirect all HTTP requests to HTTPS and disable weak protocols and cipher suites.

Example: Strict-Transport-Security "max-age=63072000; includeSubDomains; preload" alongside a 301 HTTP-to-HTTPS redirect.

low X-Content-Type-Options Header Missing #sec-65c7d57e32c0b667

zap · uat · <https://crew-f8-usva-uat.roiscloud.com/altair/assets/index-Bq01CLef.css> · status: new · discovered: 2026-08-23

CWE: CWE-693 · **OWASP 2021:** · **PCI DSS:**

<p>The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.</p>

Detailed Description:

Responses omit a Content-Security-Policy and related hardening headers (for example X-Content-Type-Options), so the browser falls back to permissive defaults. Any injection or script-loading weakness is amplified because the browser will execute attacker-supplied scripts and styles, and without nosniff a browser may reinterpret a non-HTML response (JSON, images) as a document, turning otherwise benign content into a script-execution vector.

Solution:

Set a strict Content-Security-Policy that constrains default-src, script-src, object-src and style-src and disallows inline and eval where feasible; add X-Content-Type-Options: nosniff. Apply the headers to every response, not only selected routes, and roll out with Content-Security-Policy-Report-Only first to detect breakage.

Example: Header set Content-Security-Policy "default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none'"; Header set X-Content-Type-Options "nosniff".

low Private IP Disclosure #sec-11d5d795668d4336

zap · sit · <https://crew-f8-usva-sit.roiscloud.com/altair/assets/index-BC9lUwIE.js> · status: new · discovered: 2026-08-23

CWE: CWE-497 · **OWASP 2021:** · **PCI DSS:**

<p>A private IP (such as 10.x.x.x, 172.x.x.x, 192.168.x.x) or an Amazon EC2 private hostname (for example, ip-10-0-56-78) has been found in the HTTP response body. This information might be helpful for further attacks targeting internal systems.</p>

Detailed Description:

The application discloses internal network addresses (RFC 1918 private IPs such as 10.x.x.x or 192.168.x.x) in responses, error messages, stack traces, or page source. An attacker uses the leaked addresses to map the internal network, infer infrastructure topology, and target internal services that would otherwise be unreachable or firewalled.

Solution:

Remove internal addresses and hostnames from all responses, error pages, stack traces and logs; return generic error identifiers; mask internal addresses behind the public hostname; ensure load balancer and monitoring headers do not leak backend IPs.

Example: Return { "error": "internal_error", "ref": "a1b2c3" } instead of a stack trace containing 10.0.4.12; strip backend IP headers at the proxy.

low Strict-Transport-Security Multiple Header Entries (Non-compliant with Spec) #sec-bd77fd8517b33d6b

zap · sit · <https://crew-f8-usva-sit.roiscloud.com/live/> · status: new · discovered: 2026-08-23

CWE: CWE-319 · **OWASP 2021:** · **PCI DSS:**

<p>HTTP Strict Transport Security (HSTS) headers were found, a response with multiple HSTS header entries is not compliant with the specification (RFC 6797) and only the first HSTS header will be processed others will be ignored by user agents or the HSTS policy may be incorrectly applied.</p><p>HTTP Strict Transport Security (HSTS) is a web security policy mechanism whereby a web server declares that complying user agents (such as a web browser) are to interact with it using only secure HTTPS connections (i.e. HTTP layered over TLS/SSL).</p>

Detailed Description:

Responses do not include the Strict-Transport-Security (HSTS) header, so a client that first reaches the site over HTTP - or falls back to cleartext within a mixed-content context - can be downgraded or stripped by an on-path attacker. Without HSTS the browser neither forces HTTPS on later visits nor refuses invalid certificates, leaving session credentials and data exposed to interception.

Solution:

Serve all traffic over TLS and send Strict-Transport-Security from every HTTPS response (max-age of at least six months, includeSubDomains, and preload once subdomains are verified). Redirect all HTTP requests to HTTPS and disable weak protocols and cipher suites.

Example: Strict-Transport-Security "max-age=63072000; includeSubDomains; preload" alongside a 301 HTTP-to-HTTPS redirect.

low X-Content-Type-Options Header Missing #sec-940f422c863dc4c2

zap · sit · <https://crew-f8-usva-sit.roiscloud.com/altair/assets/index-BMeTqShV.css> · status: new · discovered: 2026-08-23

CWE: CWE-693 · **OWASP 2021:** · **PCI DSS:**

<p>The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.</p>

Detailed Description:

Responses omit a Content-Security-Policy and related hardening headers (for example X-Content-Type-Options), so the browser falls back to permissive defaults. Any injection or script-loading weakness is amplified because the browser will execute attacker-supplied scripts and styles, and without nosniff a browser may reinterpret a non-HTML response (JSON, images) as a document, turning otherwise benign content into a script-execution vector.

Solution:

Set a strict Content-Security-Policy that constrains default-src, script-src, object-src and style-src and disallows inline and eval where feasible; add X-Content-Type-Options: nosniff. Apply the headers to every response, not only selected routes, and roll out with Content-Security-Policy-Report-Only first to detect breakage.

Example: Header set Content-Security-Policy "default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none'"; Header set X-Content-Type-Options "nosniff".

info **Open port 22/tcp (ssh)** #sec-cef5a33cac7380a8

nmap · sit · crew-f8-usva-sit.roiscloud.com:22 · status: confirmed · discovered: 2026-08-13

Host crew-f8-usva-sit.roiscloud.com exposes tcp port 22 (ssh). Verify this exposure is intentional and restricted by firewall or security groups.

Detailed Description:

The target exposes SSH (port 22/tcp) to the internet. SSH is a standard management service, but public exposure widens the brute-force and credential-stuffing surface and is only acceptable when policy requires a public management channel.

Solution:

Confirm SSH exposure is policy-compliant; otherwise restrict port 22 to a management allowlist or VPN, enforce key-based authentication, disable password login for privileged accounts, and rate-limit and monitor authentication failures.

Example: Firewall rule allowing 22/tcp only from the management IP range; sshd configuration with PasswordAuthentication no for admin users.

info **/live/ missing CSP, COEP, Permissions-Policy despite @fastify/helmet defaults.** #sec-632eeafd562b7f4f

pentagi · sit · /live/ · status: confirmed · discovered: 2026-08-13

/live/ missing CSP, COEP, Permissions-Policy despite @fastify/helmet defaults.

Detailed Description:

The /live/ responses rely on the default @fastify/helmet header set and lack an explicit Content-Security-Policy, cross-origin isolation headers (COEP, CORP), and a Permissions-Policy. The permissive baseline amplifies the impact of any script-injection weakness and leaves powerful browser features (camera, microphone, geolocation) available to the page.

Solution:

Add an explicit Content-Security-Policy, cross-origin isolation headers (Cross-Origin-Opener-Policy, Cross-Origin-Embedder-Policy, Cross-Origin-Resource-Policy), and a Permissions-Policy that denies sensitive features.

Example: helmet({ contentSecurityPolicy: { directives: { defaultSrc: ['self'] } }, crossOriginEmbedderPolicy: true }) in the Fastify server.

info **Re-examine Cache-control Directives** #sec-0ad3020597170e0a

zap · sit · https://crew-f8-usva-sit.roiscloud.com/engine/ · status: confirmed · discovered: 2026-08-14

CWE: CWE-525 · **OWASP 2021:** · **PCI DSS:**

<p>The cache-control header has not been set properly or is missing, allowing the browser and proxies to cache content. For static assets like css, js, or image files this might be intended, however, the resources should be reviewed to ensure that no sensitive content will be cached.</p>

Detailed Description:

Responses that carry credentials, personal data or authenticated content are sent without Cache-Control: no-store, so shared proxies, browser caches or intermediaries may store and later replay them - to other users on shared machines, or after a user logs out. Sensitive data can also linger on disk, enabling local theft.

Solution:

Set Cache-Control: no-store (and Pragma: no-cache for legacy intermediaries) on every response containing credentials, personal data or authenticated content; cache only explicitly public, non-sensitive assets such as static bundles; use Vary appropriately where caches are permitted.

Example: reply.header('Cache-Control', 'no-store') on authenticated API responses; static assets may use Cache-Control: public, max-age=31536000, immutable.

info **Open port 22/tcp (ssh)** #sec-573c0128cb1405e9

nmap · sit · 47.89.181.217:22 · status: confirmed · discovered: 2026-08-14

Host 47.89.181.217 exposes tcp port 22 (ssh). Verify this exposure is intentional and restricted by firewall or security groups.

Detailed Description:

The target exposes SSH (port 22/tcp) to the internet. SSH is a standard management service, but public exposure widens the brute-force and credential-stuffing surface and is only acceptable when policy requires a public management channel.

Solution:

Confirm SSH exposure is policy-compliant; otherwise restrict port 22 to a management allowlist or VPN, enforce key-based authentication, disable password login for privileged accounts, and rate-limit and monitor authentication failures.

Example: Firewall rule allowing 22/tcp only from the management IP range; sshd configuration with PasswordAuthentication no for admin users.

info **Information Disclosure - Suspicious Comments** #sec-55562066bf74b89d

zap · sit · <https://crew-f8-usva-sit.roiscloud.com/altair/assets/index-iOdICZWR.js> · status: confirmed · discovered: 2026-08-16

CWE: CWE-615 · **OWASP 2021:** · **PCI DSS:**

<p>The response appears to contain suspicious comments which may help an attacker.</p>

Detailed Description:

The affected component exhibits the reported weakness, increasing exposure to the corresponding vulnerability class. The precise trigger depends on how the component processes untrusted input or configuration; refer to the request/response evidence in this card for the observed behavior.

Solution:

Review the affected component against security best practices, apply the corresponding fix, and confirm the issue no longer reproduces on a follow-up scan.

Example: After applying a fix, re-run the weekly scan and verify this finding transitions to verified (no longer reproduced) rather than reopening.

info **Information Disclosure - Suspicious Comments** #sec-ec21d8658c9246fa

zap · uat · <https://crew-f8-usva-uat.roiscloud.com/altair/assets/index-B0tMGYa5.js> · status: new · discovered: 2026-08-23

CWE: CWE-615 · **OWASP 2021:** · **PCI DSS:**

<p>The response appears to contain suspicious comments which may help an attacker.</p>

Detailed Description:

The affected component exhibits the reported weakness, increasing exposure to the corresponding vulnerability class. The precise trigger depends on how the component processes untrusted input or configuration; refer to the request/response evidence in this card for the observed behavior.

Solution:

Review the affected component against security best practices, apply the corresponding fix, and confirm the issue no longer reproduces on a follow-up scan.

Example: After applying a fix, re-run the weekly scan and verify this finding transitions to verified (no longer reproduced) rather than reopening.

zap · sit · <https://crew-f8-usva-sit.roiscloud.com/altair/assets/index-BC9lUwIE.js> · status: new · discovered: 2026-08-23

CWE: CWE-615 · **OWASP 2021:** · **PCI DSS:**

<p>The response appears to contain suspicious comments which may help an attacker.</p>

Detailed Description:

The affected component exhibits the reported weakness, increasing exposure to the corresponding vulnerability class. The precise trigger depends on how the component processes untrusted input or configuration; refer to the request/response evidence in this card for the observed behavior.

Solution:

Review the affected component against security best practices, apply the corresponding fix, and confirm the issue no longer reproduces on a follow-up scan.

Example: After applying a fix, re-run the weekly scan and verify this finding transitions to verified (no longer reproduced) rather than reopening.

Fix Progress & Retest Verification

Status distribution: confirmed (23) · verified (12) · new (8)

CWE: CWE-79 · **OWASP 2021:** A03 · **PCI DSS:** 6.5.7

Cross-site scripting (XSS) is an attack technique that involves echoing attacker-supplied code into a user's browsing context.

Detailed Description:

The application reflects user-controlled input (query parameters, path segments, or form fields) directly into the HTML response without sanitization or context-aware encoding. An attacker crafts a link such as `https://<host>/live/?q=<script>alert(document.cookie)</script>` and tricks a victim into opening it; the browser then executes the injected script in the application origin, enabling session-cookie theft or privileged actions performed as the victim.

Request

GET /live/?q=test HTTP/1.1

Response

HTTP/1.1 200 OK

Solution:

Never reflect unvalidated input. Apply context-aware output encoding for every insertion point (HTML element, attribute, JavaScript, URL), validate and allowlist input against a strict schema, and set a Content-Security-Policy that disallows inline scripts. Prefer frameworks that auto-escape by default.

Example: Replace an echoing 404 handler with a static response: `reply.code(404).send({ code: 404, data: null, message: 'Not Found' });` in a template, render user data with the framework's auto-escaping syntax (`{text}` in React, `{{ value }}` in most template engines) rather than raw string concatenation.

Remediation applied: Replace the reflective 404 error handler with a static response that does not echo the request URL or query string. Return a fixed JSON body and set Content-Type: application/json so reflected input cannot be interpreted as HTML/JavaScript.

【136a5aef】 [sec-3ade3390cace96c9] high: Cross Site Scripting (Reflected)

```
+ await server.register(notFoundPlugin)
+ export default fp(async function notFoundPlugin(fastify: FastifyInstance) {
+   fastify.setNotFoundHandler((_request, reply) => {
+     reply.code(404).send({ code: 404, data: null, message: 'Not Found' })
+   })
+ }
```

Commit: 136a5aef

Retest: passed on 2026-08-16

Issue: <https://github.com/yuanzhu-ai/rois-ai/issues/11>

medium

Missing X-Frame-Options #sec-f838f93c10f395cc

zap · sit · <https://crew-f8-usva-sit.roiscloud.com/altair/> · status: verified · discovered: 2026-08-13

CWE: CWE-1021 · **OWASP 2021:** · **PCI DSS:**

The X-Frame-Options header is missing.

Detailed Description:

Responses do not carry X-Frame-Options or the frame-ancestors Content-Security-Policy directive, so the page can be embedded in a third-party <iframe>. An attacker overlays transparent elements or uses CSS to lure the victim into clicking buttons or submitting forms inside the framed page (clickjacking / UI redress), triggering sensitive actions such as roster or privilege changes without the user's intent.

Request

GET /altair/ HTTP/1.1

Response

HTTP/1.1 200 OK

Solution:

Send X-Frame-Options: DENY on every response, or the modern Content-Security-Policy frame-ancestors 'none' directive (CSP takes precedence when both are present). Only allow framing for domains with a genuine requirement.

Example: In nginx: `add_header X-Frame-Options "DENY" always;`. In Fastify/Express: `helmet.frameguard({ action: 'deny' })` or `app.use(helmet())` which sets it by default.

Remediation applied: Add the X-Frame-Options: DENY response header to prevent the application from being rendered inside a cross-origin frame.

【a4e686b7】 [sec-f838f93c10f395cc] medium: Missing X-Frame-Options

```
+ add_header X-Frame-Options "DENY" always;
+ add_header X-Content-Type-Options "nosniff" always;
+ add_header Referrer-Policy "strict-origin-when-cross-origin" always;
+ add_header Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data: blob:; font-src 'self' data:; connect-src 'self' ws: wss:; object-src 'none'; frame-ancestors 'none'; base-uri 'self'; form-action 'self';" always;
```

Commit: a4e686b7

Retest: passed on 2026-08-16

Issue: <https://github.com/yuanzhu-ai/rois-ai/issues/15>

high UAT /engine/ API authentication gap (merged C1+CA4) #sec-c0ef57bdf35c2bd

pentagi · uat · <https://crew-f8-usva-uat.roiscloud.com/engine/api/system/info> · status: verified · discovered: 2026-08-13

UAT /engine/api/system/info → 200 {"service":"ROIS Optimizer Server",...} + access-control-allow-origin: * vs SIT same endpoint → 401
www-authenticate: Bearer {"detail":"Invalid authentication credentials"}; affected <https://crew-f8-usva-uat.roiscloud.com/engine/api/system/info> (and /api/optimizers, /api/tasks/running, /api/tasks/all, /api/optimize/status|result|input|progress/{task_id}).

Detailed Description:

The engine API is reachable without authentication, so any party that can reach the service can invoke scheduling-engine operations: reading configuration and model data, triggering optimization runs, or altering state. Because the service is exposed on the UAT domain, the gap is reachable externally rather than limited to the trusted network.

Solution:

Enforce authentication on every engine endpoint, validate a token or session on each request, disable auto-generated documentation until access is controlled, restrict the service to an internal allowlist, and log and monitor access for anomalies.

Example: Require a bearer token or a reverse-proxy authentication layer on all /engine/* routes and set openapi_url=None, docs_url=None, redoc_url=None until the surface is locked down.

Remediation applied: Enforce authentication on all engine APIs by disabling the auto-generated OpenAPI, ReDoc and Swagger endpoints and removing the wildcard CORS origin. Require a valid token on every route and allowlist expected origins only.

[e40e8111] [sec-c0ef57bdf35c2bd] high: UAT /engine/ API authentication gap (merged C1+CA4)

```
+ docs_url: null
+ redoc_url: null
+ openapi_url: null
cors:
- allow_origins:
- - "*"
+ allow_origins: []
allow_methods:
app.mount("/metrics", metrics_app)
- _cors_config = config_manager.get_config().server.cors
app.add_middleware(
    CORSMiddleware,
    allow_credentials=False, # allow_origins=["*"] False
)
```

Commit: e40e8111

Retest: passed on 2026-08-16

Issue: <https://github.com/yuanzhu-ai/rois-ai/issues/12>

pentagi · sit · wss://crew-f8-usva-sit.roiscloud.com/live/ws/locks · status: verified · discovered: 2026-08-13

wss://crew-f8-usva-sit.roiscloud.com/live/ws/locks and wss://crew-f8-usva-uat.roiscloud.com/live/ws/locks accept 101 with foreign Origin https://evil.example, no cookies/auth/frames; Sec-WebSocket-Accept s3pPLMBiTxaQ9kYGzzhZRbK+xOo=.

Detailed Description:

The WebSocket endpoint accepts handshakes from any origin. Browsers do not enforce the same-origin policy for WebSocket connections, so a malicious page the victim visits can open a connection to the same service and inherit the victim's session (Cross-Site WebSocket Hijacking), pushing or reading lock state on the victim's behalf.

Solution:

Validate the Origin header on the WebSocket handshake against an allowlist of trusted origins and reject everything else, bind the channel to an authenticated session, and re-verify the user after the handshake since the browser does not authenticate the WebSocket upgrade.

Example: if (!isAllowedWsOrigin(request.headers.origin)) reply.code(403).send({ error: "forbidden origin" }).

Remediation applied: Validate the Origin header on the WebSocket handshake for //live/ws/locks and reject connections from unexpected origins, so a malicious site cannot open a cross-site WebSocket to the locks channel.

【22004ac4】 [sec-be2d77b2278d5be9] high: CSWSH //live/ws/locks (merged C5+CA3)

```
+ export const isAllowedWsOrigin = (origin: string | undefined, host: string | undefined): boolean => {  
+   if (!origin) return true  
+   try {  
+     const originUrl = new URL(origin)
```

Commit: 22004ac4

Retest: passed on 2026-08-16

Issue: <https://github.com/yuanzhu-ai/rois-ai/issues/13>

high Unauthenticated FastAPI OpenAPI spec #sec-9023d758cf2ab0b6

pentagi · sit · /engine/openapi.json · status: verified · discovered: 2026-08-13

SIT+UAT /engine/openapi.json → 200, 18,096 B, OpenAPI 3.1.0, 16 paths incl. destructive POST/DELETE/PUT routes; byte-identical SIT vs UAT.

Detailed Description:

The FastAPI OpenAPI schema (openapi.json) is served without authentication, disclosing the complete API surface - every route, parameter, model and operation - to anyone who can reach the service. This metadata is a ready-made attack map for the engine API.

Solution:

Disable the schema in production with openapi_url=None, or gate it behind authentication and an internal-only network allowlist.

Example: FastAPI(..., openapi_url=None).

Remediation applied: Disable the auto-generated FastAPI OpenAPI schema by setting openapi_url to null in the FastAPI app configuration, preventing unauthenticated disclosure of the API surface.

【e40e8111】 [sec-9023d758cf2ab0b6] high: Unauthenticated FastAPI OpenAPI spec

```
+ docs_url: null
+ redoc_url: null
+ openapi_url: null
  cors:
-   allow_origins:
-     - "*"
+   allow_origins: []
+   allow_methods:
+   app.mount("/metrics", metrics_app)
-   _cors_config = config_manager.get_config().server.cors
+   app.add_middleware(
+     CORSMiddleware,
+     allow_credentials=False, # allow_origins=["*"] False
+   )
```

Commit: e40e8111

Retest: passed on 2026-08-16

Issue: <https://github.com/yuanzhu-ai/rois-ai/issues/14>

medium

Unauthenticated Swagger UI /engine/docs (200, 1,023 B, swagger-ui-dist@5). #sec-57388404fa80cfd7

pentagi · sit · /engine/docs · status: verified · discovered: 2026-08-13

Unauthenticated Swagger UI /engine/docs (200, 1,023 B, swagger-ui-dist@5).

Detailed Description:

FastAPI auto-generates interactive documentation - the Swagger UI at /docs, ReDoc at /redoc and the underlying OpenAPI schema - and serves it without authentication. Anyone who can reach the service can enumerate every endpoint, parameter and data shape of the engine API, turning the schema into a roadmap for targeted attacks.

Solution:

Disable auto-generated documentation in non-development deployments (openapi_url=None, docs_url=None, redoc_url=None) or protect it behind authentication; keep the schema internal to the engineering team.

Example: FastAPI(..., openapi_url=None, docs_url=None, redoc_url=None).

Remediation applied: Disable the auto-generated Swagger UI by setting docs_url to null in the FastAPI app configuration so the interactive API documentation is not publicly reachable.

[e40e8111] [sec-57388404fa80cfd7] medium: Unauthenticated Swagger UI /engine/docs (200, 1,023 B, swagger-ui-dist@5).

```
+ docs_url: null
+ redoc_url: null
+ openapi_url: null
cors:
- allow_origins:
- - "*"
+ allow_origins: []
allow_methods:
app.mount("/metrics", metrics_app)
- _cors_config = config_manager.get_config().server.cors
app.add_middleware(
    CORSMiddleware,
    allow_credentials=False, # allow_origins=["*"] False
)
```

Commit: e40e8111

Retest: passed on 2026-08-16

Issue: <https://github.com/yuanzhu-ai/rois-ai/issues/16>

medium

Unauthenticated ReDoc UI /engine/redoc (200, 905 B). #sec-75f507a27694b07d

pentagi · sit · /engine/redoc · status: verified · discovered: 2026-08-13

Unauthenticated ReDoc UI /engine/redoc (200, 905 B).

Detailed Description:

FastAPI auto-generates interactive documentation - the Swagger UI at /docs, ReDoc at /redoc and the underlying OpenAPI schema - and serves it without authentication. Anyone who can reach the service can enumerate every endpoint, parameter and data shape of the engine API, turning the schema into a roadmap for targeted attacks.

Solution:

Disable auto-generated documentation in non-development deployments (openapi_url=None, docs_url=None, redoc_url=None) or protect it behind authentication; keep the schema internal to the engineering team.

Example: FastAPI(..., openapi_url=None, docs_url=None, redoc_url=None).

Remediation applied: Disable the auto-generated ReDoc UI by setting redoc_url to null in the FastAPI app configuration so the interactive API documentation is not publicly reachable.

【e40e8111】 [sec-75f507a27694b07d] medium: Unauthenticated ReDoc UI /engine/redoc (200, 905 B).

```
+ docs_url: null
+ redoc_url: null
+ openapi_url: null
cors:
- allow_origins:
- - "*"
+ allow_origins: []
allow_methods:
app.mount("/metrics", metrics_app)
- _cors_config = config_manager.get_config().server.cors
app.add_middleware(
    CORSMiddleware,
    allow_credentials=False, # allow_origins=["*"] False
)
```

Commit: e40e8111

Retest: passed on 2026-08-16

Issue: <https://github.com/yuanzhu-ai/rois-ai/issues/17>

medium

Wildcard CORS on engine service (Starlette CORSMiddleware) #sec-67393692463874b2

pentagi · sit · status: verified · discovered: 2026-08-13

access-control-allow-origin: * for any Origin; preflight 200 text/plain "OK", allow-methods GET, POST, allow-headers incl. Authorization/X-API-Key/X-Airline, max-age 600.

Detailed Description:

The engine service answers cross-origin requests with Access-Control-Allow-Origin: * (the Starlette CORSMiddleware default), so any website can read responses from the engine API in a victim's browser context. If the API returns data or accepts state-changing calls, a malicious site can exfiltrate data or trigger actions cross-origin.

Solution:

Replace the wildcard with an explicit allowlist of the expected application origins, do not echo the Origin header, keep allow_credentials=False (browsers reject it combined with *), and restrict methods to the minimal set.

Example: CORSMiddleware(allow_origins=["https://crew-f8-usva-sit.roiscloud.com", "https://crew-f8-usva-uat.roiscloud.com"], allow_methods=["GET", "POST"], allow_credentials=False).

Remediation applied: Replace the wildcard CORS origin with an explicit allowlist of expected origins, or disable CORS entirely where the engine is not consumed cross-origin.

[e40e8111] [sec-67393692463874b2] medium: Wildcard CORS on engine service (Starlette CORSMiddleware)

```
+ docs_url: null
+ redoc_url: null
+ openapi_url: null
cors:
- allow_origins:
- - "*"
+ allow_origins: []
allow_methods:
app.mount("/metrics", metrics_app)
- _cors_config = config_manager.get_config().server.cors
app.add_middleware(
CORSMiddleware,
allow_credentials=False, # allow_origins=["*"] False
)
```

Commit: e40e8111

Retest: passed on 2026-08-16

Issue: <https://github.com/yuanzhu-ai/rois-ai/issues/18>

medium

Content Security Policy (CSP) Header Not Set #sec-81bb353a41ba97f1

zap · uat · <https://crew-f8-usva-uat.roiscloud.com/altair/> · status: verified · discovered: 2026-08-14

CWE: CWE-693 · **OWASP 2021:** · **PCI DSS:**

<p>Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.</p>

Detailed Description:

Responses omit a Content-Security-Policy and related hardening headers (for example X-Content-Type-Options), so the browser falls back to permissive defaults. Any injection or script-loading weakness is amplified because the browser will execute attacker-supplied scripts and styles, and without nosniff a browser may reinterpret a non-HTML response (JSON, images) as a document, turning otherwise benign content into a script-execution vector.

Solution:

Set a strict Content-Security-Policy that constrains default-src, script-src, object-src and style-src and disallows inline and eval where feasible; add X-Content-Type-Options: nosniff. Apply the headers to every response, not only selected routes, and roll out with Content-Security-Policy-Report-Only first to detect breakage.

Example: Header set Content-Security-Policy "default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none'"; Header set X-Content-Type-Options "nosniff".

Remediation applied: Add a Content Security Policy response header that restricts script, style, and frame sources to trusted origins, and verify the application still loads with the policy applied.

[a4e686b7] [sec-81bb353a41ba97f1] medium: Content Security Policy (CSP) Header Not Set

```
+ add_header X-Frame-Options "DENY" always;
+ add_header X-Content-Type-Options "nosniff" always;
+ add_header Referrer-Policy "strict-origin-when-cross-origin" always;
+ add_header Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data: blob;; font-src 'self' data;; connect-src 'self' ws: wss;; object-src 'none'; frame-ancestors 'none'; base-uri 'self'; form-action 'self';" always;
```

Commit: a4e686b7

Retest: passed on 2026-08-16

Issue: <https://github.com/yuanzhu-ai/rois-ai/issues/19>

CWE: CWE-1021 · **OWASP 2021:** · **PCI DSS:**

<p>The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.</p>

Detailed Description:

Responses do not carry X-Frame-Options or the frame-ancestors Content-Security-Policy directive, so the page can be embedded in a third-party <iframe>. An attacker overlays transparent elements or uses CSS to lure the victim into clicking buttons or submitting forms inside the framed page (clickjacking / UI redress), triggering sensitive actions such as roster or privilege changes without the user's intent.

Solution:

Send X-Frame-Options: DENY on every response, or the modern Content-Security-Policy frame-ancestors 'none' directive (CSP takes precedence when both are present). Only allow framing for domains with a genuine requirement.

Example: In nginx: add_header X-Frame-Options "DENY" always;. In Fastify/Express: helmet.frameguard({ action: 'deny' }) or app.use(helmet()) which sets it by default.

Remediation applied: Add the X-Frame-Options: DENY response header (or the frame-ancestors Content-Security-Policy directive) to prevent clickjacking and UI redressing.

【a4e686b7】 [sec-cc3eaa987ac95c7e] medium: Missing Anti-clickjacking Header

```
+ add_header X-Frame-Options "DENY" always;
+ add_header X-Content-Type-Options "nosniff" always;
+ add_header Referrer-Policy "strict-origin-when-cross-origin" always;
+ add_header Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data: blob;; font-src 'self' data;; connect-src 'self' ws: wss;; object-src 'none'; frame-ancestors 'none'; base-uri 'self'; form-action 'self';" always;
```

Commit: a4e686b7

Retest: passed on 2026-08-16

Issue: <https://github.com/yuanzhu-ai/rois-ai/issues/20>

medium

Content Security Policy (CSP) Header Not Set #sec-e876bb78459508c4

zap · sit · <https://crew-f8-usva-sit.roiscloud.com/altair/> · status: verified · discovered: 2026-08-14

CWE: CWE-693 · **OWASP 2021:** · **PCI DSS:**

<p>Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.</p>

Detailed Description:

Responses omit a Content-Security-Policy and related hardening headers (for example X-Content-Type-Options), so the browser falls back to permissive defaults. Any injection or script-loading weakness is amplified because the browser will execute attacker-supplied scripts and styles, and without nosniff a browser may reinterpret a non-HTML response (JSON, images) as a document, turning otherwise benign content into a script-execution vector.

Solution:

Set a strict Content-Security-Policy that constrains default-src, script-src, object-src and style-src and disallows inline and eval where feasible; add X-Content-Type-Options: nosniff. Apply the headers to every response, not only selected routes, and roll out with Content-Security-Policy-Report-Only first to detect breakage.

Example: Header set Content-Security-Policy "default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none'"; Header set X-Content-Type-Options "nosniff".

Remediation applied: Add a Content Security Policy response header that restricts script, style, and frame sources to trusted origins, and verify the application still loads with the policy applied.

【a4e686b7】 [sec-e876bb78459508c4] medium: Content Security Policy (CSP) Header Not Set

```
+ add_header X-Frame-Options "DENY" always;
+ add_header X-Content-Type-Options "nosniff" always;
+ add_header Referrer-Policy "strict-origin-when-cross-origin" always;
+ add_header Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data: blob;; font-src 'self' data;; connect-src 'self' ws: wss;; object-src 'none'; frame-ancestors 'none'; base-uri 'self'; form-action 'self';" always;
```

Commit: a4e686b7

Retest: passed on 2026-08-16

Issue: <https://github.com/yuanzhu-ai/rois-ai/issues/21>

CWE: CWE-1021 · **OWASP 2021:** · **PCI DSS:**

<p>The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.</p>

Detailed Description:

Responses do not carry X-Frame-Options or the frame-ancestors Content-Security-Policy directive, so the page can be embedded in a third-party <iframe>. An attacker overlays transparent elements or uses CSS to lure the victim into clicking buttons or submitting forms inside the framed page (clickjacking / UI redress), triggering sensitive actions such as roster or privilege changes without the user's intent.

Solution:

Send X-Frame-Options: DENY on every response, or the modern Content-Security-Policy frame-ancestors 'none' directive (CSP takes precedence when both are present). Only allow framing for domains with a genuine requirement.

Example: In nginx: add_header X-Frame-Options "DENY" always;. In Fastify/Express: helmet.frameguard({ action: 'deny' }) or app.use(helmet()) which sets it by default.

Remediation applied: Add the X-Frame-Options: DENY response header (or the frame-ancestors Content-Security-Policy directive) to prevent clickjacking and UI redressing.

【a4e686b7】 [sec-3930dc7acec994a1] medium: Missing Anti-clickjacking Header

```
+ add_header X-Frame-Options "DENY" always;
+ add_header X-Content-Type-Options "nosniff" always;
+ add_header Referrer-Policy "strict-origin-when-cross-origin" always;
+ add_header Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data: blob;; font-src 'self' data:; connect-src 'self' ws: wss:; object-src 'none'; frame-ancestors 'none'; base-uri 'self'; form-action 'self';" always;
```

Commit: a4e686b7

Retest: passed on 2026-08-16

Issue: <https://github.com/yuanzhu-ai/rois-ai/issues/22>

Compliance Matrix

Each finding is normalized against three industry frameworks: the Common Weakness Enumeration (CWE), the OWASP Top 10 (2021), and the PCI DSS secure-coding requirements. The CWE mapping below is the source of truth used by this assessment; every finding card also carries a Solution derived from its CWE. PCI DSS references follow the 6.5.x secure-coding requirement numbering (in PCI DSS 4.0 the equivalent engineering controls are consolidated under requirement 6.2).

CWE to OWASP Top 10 (2021) and PCI DSS mapping:

CWE	OWASP Top 10 2021	PCI DSS
CWE-79	A03	6.5.7
CWE-89	A03	6.5.1
CWE-287	A07	8.1
CWE-502	A08	6.5.8
CWE-200	A01	
CWE-611	A05	6.5.5

OWASP Top 10 (2021) categories referenced by this assessment:

ID	Category	Description
----	----------	-------------

A01	Broken Access Control	Restrictions on what authenticated users are allowed to do are not properly enforced, enabling unauthorized data access or privilege escalation (IDOR, missing function-level authorization).
A02	Cryptographic Failures	Weak or missing cryptography protecting sensitive data in transit or at rest, including outdated hashing and poor key management.
A03	Injection	Untrusted user input is executed by an interpreter (SQL, NoSQL, OS command, LDAP, template injection, cross-site scripting).
A04	Insecure Design	Design or architectural flaws that security controls at the implementation layer cannot fully mitigate.
A05	Security Misconfiguration	Insecure defaults, incomplete hardening, open cloud storage, misconfigured HTTP headers and unnecessary features (includes XXE).
A06	Vulnerable and Outdated Components	Use of components with known vulnerabilities, or outdated and unsupported software.
A07	Identification and Authentication Failures	Flaws in identity proofing, authentication and session management that allow credential theft or session hijacking.
A08	Software and Data Integrity Failures	Code and infrastructure that do not protect against integrity violations, such as unsafe deserialization and untrusted updates.
A09	Security Logging and Monitoring Failures	Failure to log security-relevant events or to detect and alert on them, hindering incident response.
A10	Server-Side Request Forgery	The server fetches a user-controlled URL, allowing access to internal resources, firewalled services and cloud metadata.

PCI DSS secure-coding requirements (6.5.x) referenced by this assessment:

Requirement	Description
6.5.1	Injection flaws, particularly SQL injection, plus OS command, LDAP and XPath injection.
6.5.2	Buffer overflows in languages that do not validate data size before writing to a memory address.
6.5.3	Insecure cryptographic storage of authentication credentials or cardholder data.
6.5.4	Insecure communications - failure to encrypt network traffic with strong cryptography (TLS).
6.5.5	Improper error handling that discloses sensitive information in error messages.
6.5.6	All high-risk vulnerabilities identified by the vulnerability identification process.
6.5.7	Cross-site scripting (XSS) - validation of all parameters and context-sensitive output encoding.
6.5.8	Improper access control - insecure direct object references, directory traversal and unrestricted access to functions.
6.5.9	Cross-site request forgery (CSRF).
6.5.10	Broken authentication and session management weaknesses in credentials and session cookies.

Recommendations

Address the 4 critical/high severity findings first, then re-run the weekly scan to confirm each fix (retest). Fix evidence — commit, code snippet, and screenshots — is recorded per finding in the Fix Progress section.